

Algorithm 3 BasicSR / FullSR seed: a search engine with eight policy slots

Input Data D ; grammar; search budget; J populations of size N ; K inner cycles per population pass.

```
1 Initialize policy state  $S$  and random populations  $\mathcal{P} = (P_1, \dots, P_J)$ 
2 Score initial trees with Loss; call UpdateState( $\mathcal{P}, S$ )
3 while search budget and iteration allowance remain do
4   for each population  $P_j$  do
5     for  $c = 1, \dots, K$  do
6       UpdateState( $\mathcal{P}, S$ )
7       for  $e = 1, \dots, \lceil |P_j|/15 \rceil$  do
8         if  $\text{Unif}(0, 1) \leq 0.0259$  and  $|P_j| \geq 2$  then
9            $a, b \leftarrow \text{Select}(P_j, S), \text{Select}(P_j, S)$ 
10          If  $a = b$ , use the next population member as  $b$ 
11           $T \leftarrow \text{Crossover}(a, b, S)$  # Up to two trees, with their parents
12        else
13           $a \leftarrow \text{Select}(P_j, S); T \leftarrow \text{Mutate}(a, S)$ 
14           $C \leftarrow \emptyset$ 
15          for each proposed tree  $t$  and corresponding parent  $a$  do
16             $u \leftarrow \text{MAKEINDIVIDUAL}(t, \text{Loss}(t, D, S))$ 
17            If Accept( $a, u, S$ ), add  $u$  to  $C$ 
18          If  $C \neq \emptyset$ , set  $P_j \leftarrow \text{Survive}(P_j, C, S)$ 
19          Run configured simplification and constant optimization on  $P_j$ 
20          Mark the population pass complete; UpdateState( $\mathcal{P}, S$ )
21           $\mathcal{P} \leftarrow \text{UpdatePopulation}(\mathcal{P}, S)$ 
22 return the policy archive, ordered by complexity (or the best member if empty)
```

Slot	Baseline policy
Loss	MSE; cost equals loss. Invalid or extreme predictions give $+\infty$.
Select	Best of $\min(15, P)$ distinct uniformly sampled members.
Mutate	Replace a uniform node by a terminal (probability 1/2) or a shallow random operator.
Crossover	Swap uniformly chosen subtrees; up to 10 constraint-validity attempts.
Accept	Accept every constructed child.
Survive	Keep the best N from parents and children, ranked by cost.
UpdateState	Keep 10 distinct finite-loss expressions; refresh after completed passes.
UpdatePopulation	Identity: no migration.

Implementation distinction. BasicSR enables simplification and constant optimization (probability 0.14). The current FullSR seed in `SRConfig.jl` lacks those state options, so the same engine’s postprocessing hook is inactive. The eight baseline policy functions otherwise implement the same search choices. Mutation also allows up to 10 validity attempts; failed proposals produce no child.